DHBW Stuttgart

Karsten Keßler

Datenbanksysteme

2. Datenbankmodelle und -architekturen

Kurs Informatik TINF24ITA
Stand: 24.09.2026

Übersicht der Gesamtvorlesung (vorläufig)

Sem.	Nr.	Thema	Inhalte (Auszug)
3	1	Einführung	Einführung, Begriffe, Eigenschaften, Funktionen, Datenintegration
	2	Datenbankmodelle u. -architekturen	Architektur und SPARC, verschiedene Datenbankmodelle, Entwicklung
	3	ERM	Entity-Relationship-Modell, konzeptueller Entwurf, Entwurfsprozess
	4	Relationales Modell	Relationales Datenbankmodell im Detail, Schlüssel und Integritätsbedingungen, Abbildung ER → Relation
	5	Relationale Algebra	Relationale Algebren, Operatoren u. Operatorbäume, Anwendung, Nützlichkeit
	6	Normalformen	Anomalien, funktionale Abhängigkeiten, 1NF-3NF und BCNF, Dekomposition, Synthesealgorithmus
4	7	NoSQL & moderne Datenmodelle	Dokument-, Graph- und Vektordatenbanken, Einsatzszenarien
	8	Transaktionen	ACID, grundlegende Schwierigkeiten bei Mehrbenutzersystemen, gängige Lösungen
	9	SQL DML	Datenmanipulation mit SQL
	10	SQL DDL	Datendefinition mit SQL
	11	Integrität	Integrität im relationalen Modell, Umsetzung (in SQL)
	12	Sichten	Sichten (in SQL), Vor- und Nachteile, Umgang mit Sichten
	13	Indizes	Interne Strukturen (B-Bäume, Hashing), Nützlichkeit für Anfragen

Inhalte der heutigen Vorlesung



- Anforderungen
- Typen
- ANSI SPARC
- Schichten
- Relationenmodell
- Übersicht anderer Modelle

Lernziele dieses Kapitels

- Anforderungen an ein Datenbanksystem benennen
- Typen von Datenbanksystemen unterscheiden
- ANSI-SPARC-Architektur und Schichtenmodell erläutern
- Aufbau des Relationenmodells einordnen
- Weitere Datenmodelle im Überblick vergleichen

Anforderungen an eine Datenbankarchitektur

- Die Daten werden zentral verwaltet und integriert.
- Die Datenhaltung ist anwendungsunabhängig.
- Es wird ein einfaches API für den Datenzugriff bereitgestellt.
- Die Integritätskontrolle wird zentral vorgenommen.
- Parallele Verarbeitung wird ermöglicht.
- Hohe Verfügbarkeit und Fehlertoleranz wird angestrebt.
- Ein Transaktionsschutz wird integriert.

In gewachsenen IT-Landschaften von Unternehmen gibt es in aller Regel eine Vielzahl von DBMS.

→ Grund: Bei der Einführung eines neuen Anwendungssystems wird eine neue Datenbank mit eingeführt
→ Folge: Viele Schnittstellen, Redundanz, mangelnde Verfügbarkeit von Informationen, hoher Pflegeaufwand

Architektur von Datenbanksystemen

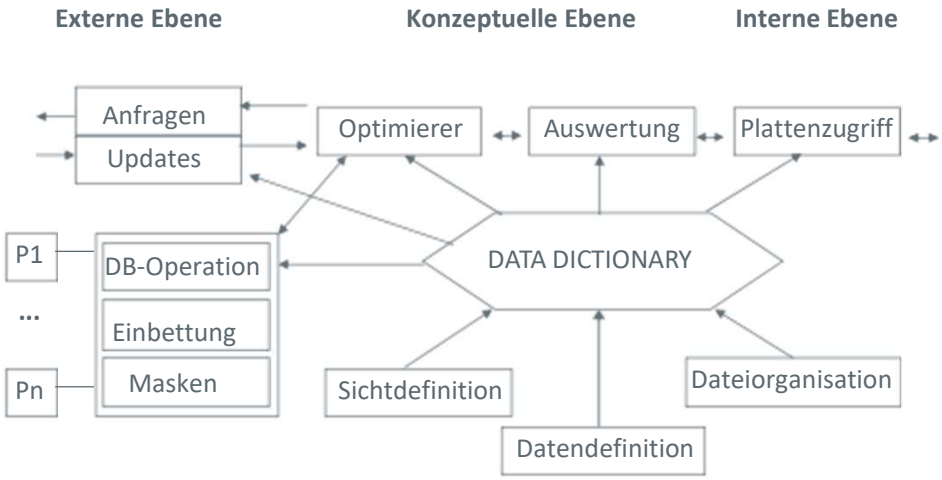
Schema Architektur

- **Konzeptuelles Schema**: Ergebnis aus Datenmodellierung (Daten und deren Beziehungen), Datenbankentwurf und Datendefinition
- **Internes (physisches) Schema**: Dateiorganisation und Zugriffspfade
- **Externes Schema**: Ergebnis aus Sichtdefinition, Sichten auf konzeptuelles Schema in Form von Forumalern, Listen, Masken usw.

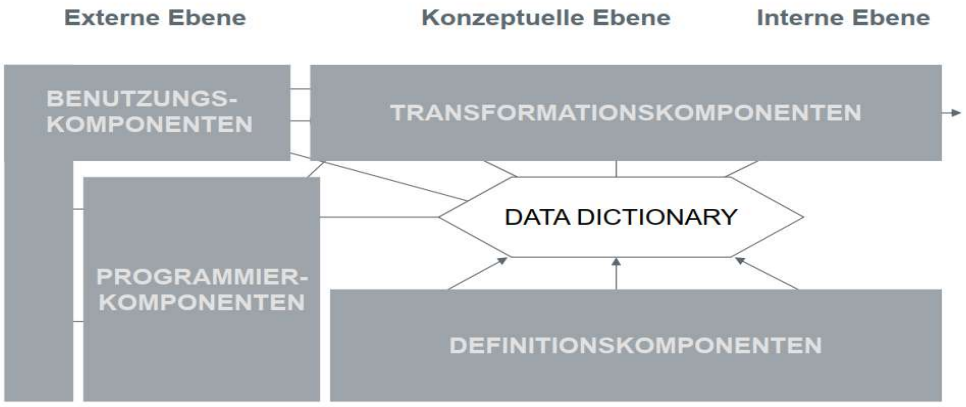
System Architektur

- **Definitionskomponenten**
- **Programmierkomponenten**
- **Benutzungskomponenten**
- **Transformationskomponenten**

ANSI SPARC Architektur



ANSI SPARC Architektur



Data Dictionary (Auszug):

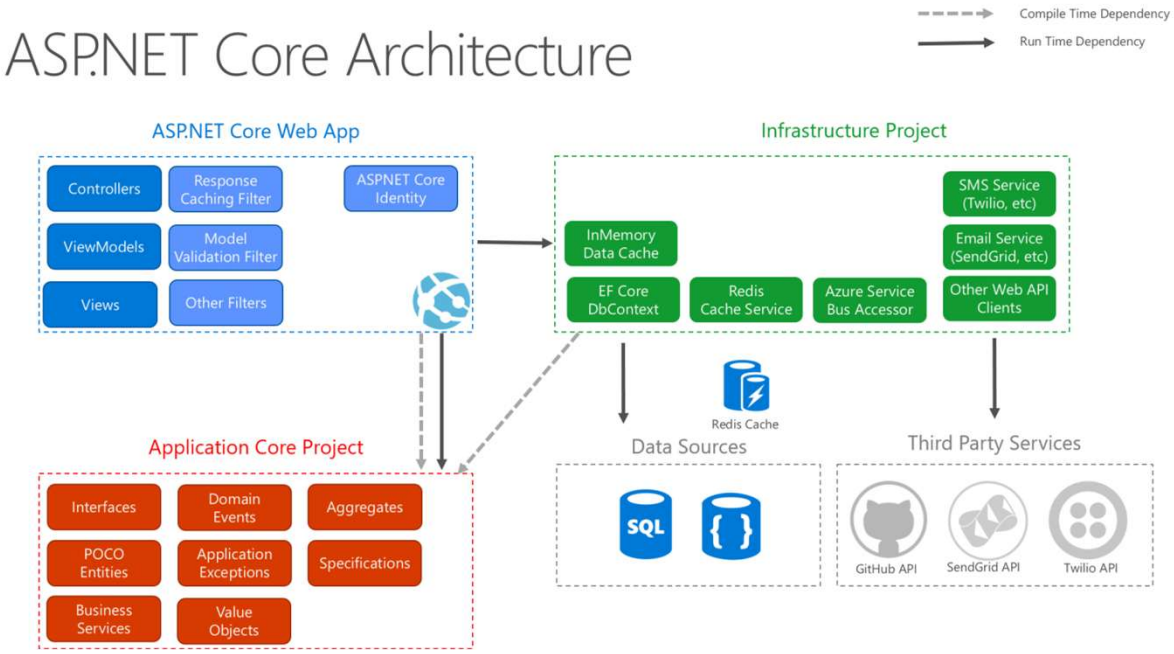
```
+---------------------------+-----------------------------------------------------------+
| Variable_name             | Value                                                     |
+---------------------------+-----------------------------------------------------------+
| basedir                   | /usr/local/mysql-5.7.17-macos10.12-x86_64/                |
| character_sets_dir        | /usr/local/mysql-5.7.17-macos10.12-x86_64/share/charsets/ |
| datadir                   | /usr/local/mysql/data/                                    |
| innodb_data_home_dir      |                                                           |
| innodb_log_group_home_dir | ./                                                        |
| innodb_tmpdir             |                                                           |
| lc_messages_dir           | /usr/local/mysql-5.7.17-macos10.12-x86_64/share/          |
| plugin_dir                | /usr/local/mysql/lib/plugin/                              |
| slave_load_tmpdir         | /var/folders/zz/zyxvpxvq6csfxvn_n000009800002_/T/         |
| tmpdir                    | /var/folders/zz/zyxvpxvq6csfxvn_n000009800002_/T/         |
+---------------------------+-----------------------------------------------------------+
```

Erzeugung des Auszugs (MySQL):

```
-- Verzeichnis-Variablen aus dem Data Dictionary anzeigen
SHOW VARIABLES LIKE '%dir%';

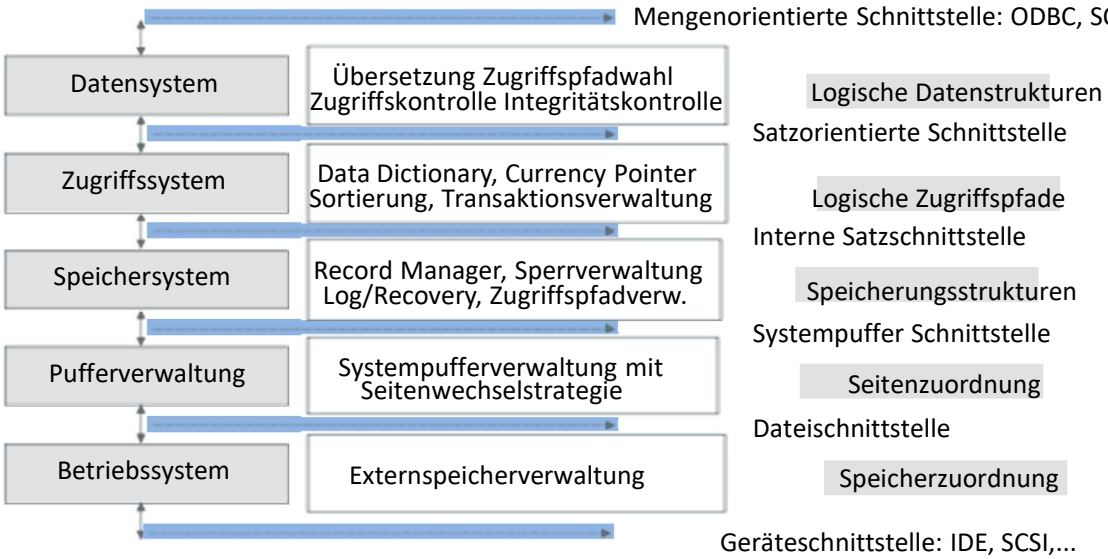
-- Alternativ über das Information Schema:
SELECT VARIABLE_NAME, VARIABLE_VALUE
  FROM performance_schema.global_variables
 WHERE VARIABLE_NAME LIKE '%dir%'
 ORDER BY VARIABLE_NAME;
```

Beispiel: Schichten-Architekturen Web

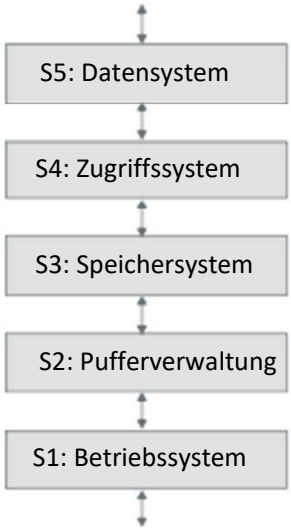


https://docs.microsoft.com/de-de/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures

5-Schichten-Architektur



5-Schichten-Architektur



	Objekte	**Operatoren**
S5: Datensystem	Relationen Sichten	SQL: Select ... From ...
S4: Zugriffssystem	Externe Sätze Indexstrukturen	FIND NEXT satz STORE satz
S3: Speichersystem	Interne Sätze Bäume, Hash-Tabellen	INSERT in B-Baum
S2: Pufferverwaltung	Segmente Seiten	Bereitstellen Seite j Freigeben Seite j
S1: Betriebssystem	Dateien Blöcke	Lese Block k Schreibe Block k
	Zylinder Spuren	Treiber

S1: Externspeicherverwaltung

- Realisierung von Dateien bestehen aus Blöcken auf physischen Geräten.
- Verwalten des physischen Speichers

S2:Systempufferverwaltung

- Realisierung eines oder mehrer Segmente bestehen aus Seiten fester Größe
- Effizientes Auffinden einer Seite im Puffer.
- Speicherzuordnung
- Pufferverwaltung
- Schattenspeicherkonzept (der Inhalt eines Segments bleibt während Sicherungsintvervall unverändert)

S3: Realisierung von Speicherstruturen

- Bereitstellung von physischen Speicherstruktuiren (vgl. ANSI/SPARC)
- Freispeicherverwaltung innerhalb von Segmenten und Seiten
- Adressierung von Sätzen innerhalb Seiten
- Abbildung von Datensätzen in Seiten

S4: Die satzorientierte Schnittstelle

- Realislierung von logischen Sätzen und Zugriffspfaden
- Bereitstellung von generischer Funktionen (Sortierung, Mittelwert)
- Transaktionsunterstützung (BEGIN WORK, COMMOT, ROLLBACK WORK)

S5: Die Mengenorientierte Schnittstelle

- Algebraische Optimierung, Anfragestrukturierung
- Codegenerierung für mengenorientierte Anforderungen
- Ausführung von DB-Anforderungen

Konkrete Datenbanken und Systemarchitekturen (RDBMS)

IMS (Information Management System), IBM, 60er Jahre

- Hierarchisches Datenbanksystem
- Zugriff ausschließlich über DB-Anwendungsprogramme
- Internes Schema im Anwendungsprogramm sichtbar

UDS (Universelles Datenbanksystem), Siemens, 1976

- Netzwerk-Datenbanksystem
- Entspricht ANSI-SPARC Grobarchitektur
- Kein echter Optimierer

DB2, ORACLE, INGRES, SYBASE, MS SQL Server

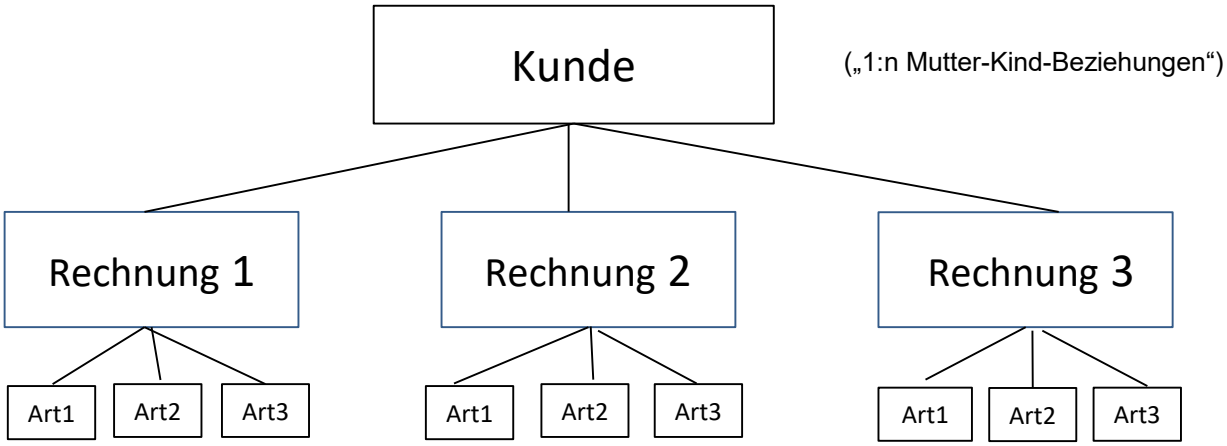
- Relationale Datenbanksysteme
- Drei-Ebenen-Architektur nach ANSI-SPARC
- Einheitliche Datenbanksprache (SQL) und Einbettung in kommerzielle Programmiersprache
- Mehrbenutzerbetrieb, Zugriffskontrolle, Datensicherheit

Pseudo-RDBS

- SQLite´Programmbibliothek DB in „einer Datei“, keine Zugriffsverwaltung
- Berkeley DB – Key Value Store, auch embedded
- MS-Access – moderne Oberfläche für RDBS, kaum kontrollierter Mehrbenutzerzugriff, kein Optimierer, Non-Standard SQL

Ein **Datenbankmodell** legt Syntax und Semantik der Datenbankbeschreibung für ein Datenbanksystem fest. Es besitzt ein System von Konzepten zur Beschreibung von Datenbanken. Letztendlich entspricht es der Struktur eines Datenbanksystems.

Hierarchisches Modell



Speicherreihenfolge der Sätze (Preorder)

Kunde
- Rechnung 1
  - Art1
  - Art2
  - Art3
- Rechnung 2
  - Art1
  - Art2
  - Art3
- Rechnung 3
  - Art1
  - Art2
  - Art3

Prinzip
Die Daten liegen in einer Baumstruktur: jeder Satz hat genau einen Vater, aber beliebig viele Kinder (1:n). Der Zugriff erfolgt navigierend von der Wurzel über die Zeiger der Struktur.

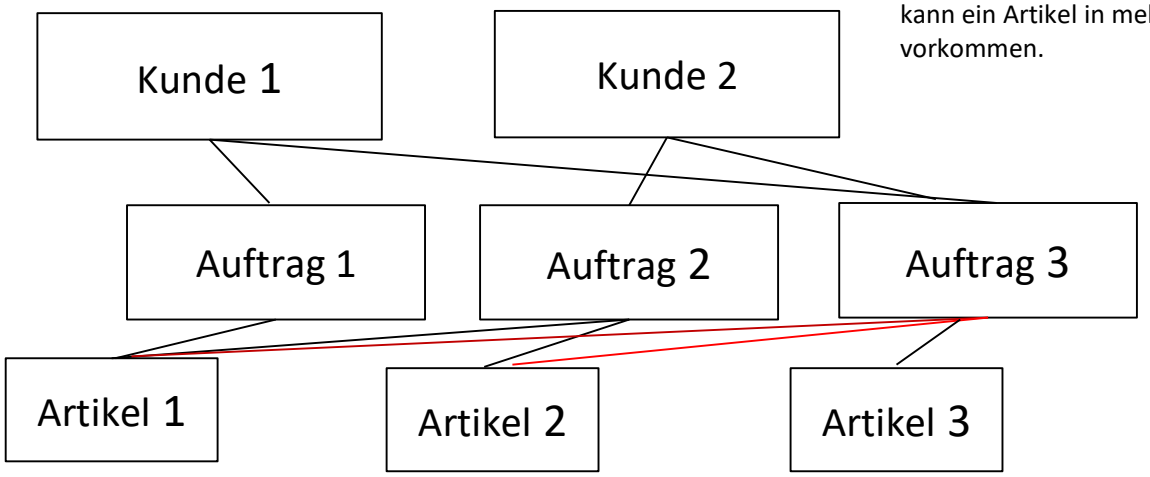
Pro
Sehr schnelle Zugriffe entlang der Hierarchie, einfaches und speichersparendes Modell.

Kontra
n:m-Beziehungen nur über Redundanz abbildbar, Strukturänderungen aufwändig, Zugriff nur über den vorgesehenen Pfad.

Typische Systeme
IBM IMS, LDAP-Verzeichnisdienste, Windows-Registry.

Netzwerkmodell

(„n:m Vater-Mutter-Kind-Beziehungen“) Das bedeutet, dass ein Kunde mehrere Aufträge haben kann und ein Auftrag mehrere Artikel enthalten kann. Ebenso kann ein Artikel in mehreren Aufträgen vorkommen.



Prinzip
Weiterentwicklung des hierarchischen Modells: ein Satz darf mehrere Vorgänger haben, Sätze sind über Zeigerketten („Sets“) verbunden. Damit lassen sich n:m-Beziehungen ohne Redundanz abbilden.
Der Zugriff bleibt navigierend und erfolgt satzweise über Befehle wie find, get und store (CODASYL-Standard).

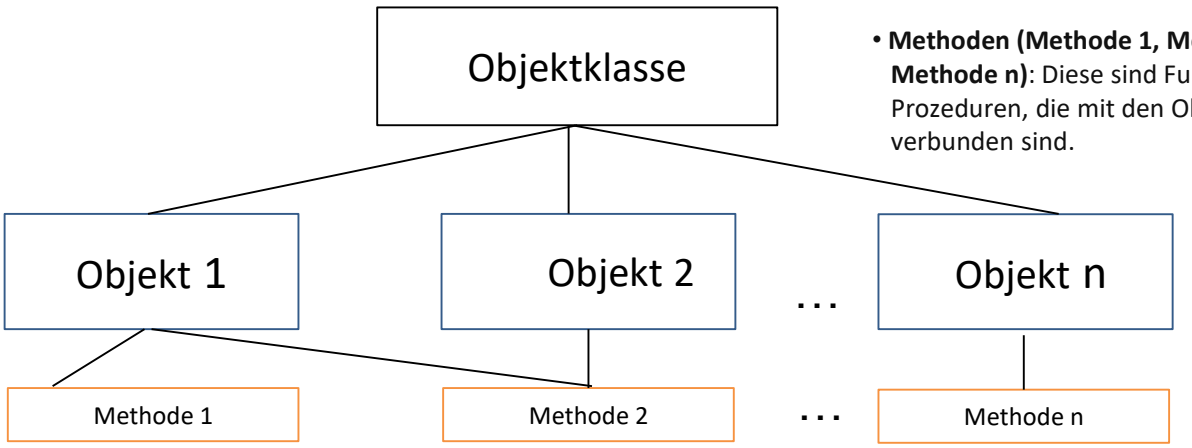
Pro
Keine starre Hierarchie, mehrere Zugriffswege auf denselben Satz, sehr speichereffizient und schnell bei bekannten Pfaden.

Kontra
Netzstrukturen werden bei großen Datenmengen unübersichtlich, Änderungen am Schema sind aufwändig, keine mengenorientierte Abfragesprache wie SQL.

Typische Systeme
CODASYL-Datenbanken wie IDMS, UDS und TOTAL – heute überwiegend in Altsystemen.

Objektorientiertes Modell

- **Objektklasse**: Dies ist die zentrale Klasse, von der die Objekte abgeleitet werden.
- **Objekte (Objekt 1, Objekt 2, … Objekt n)**: Diese repräsentieren Instanzen der Objektklasse.
- **Methoden (Methode 1, Methode 2, … Methode n)**: Diese sind Funktionen oder Prozeduren, die mit den Objekten verbunden sind.



Prinzip
Daten und Verhalten werden gemeinsam als Objekte gespeichert: Klassen, Vererbung, komplexe und verschachtelte Datentypen sowie eine eindeutige Objektidentität (OID). Objekte behalten in der Datenbank dieselbe Struktur wie im Programm, der Bruch zwischen Programmiersprache und Tabellen entfällt. Abfragen erfolgen über OQL oder direkt über die Objektsprache.

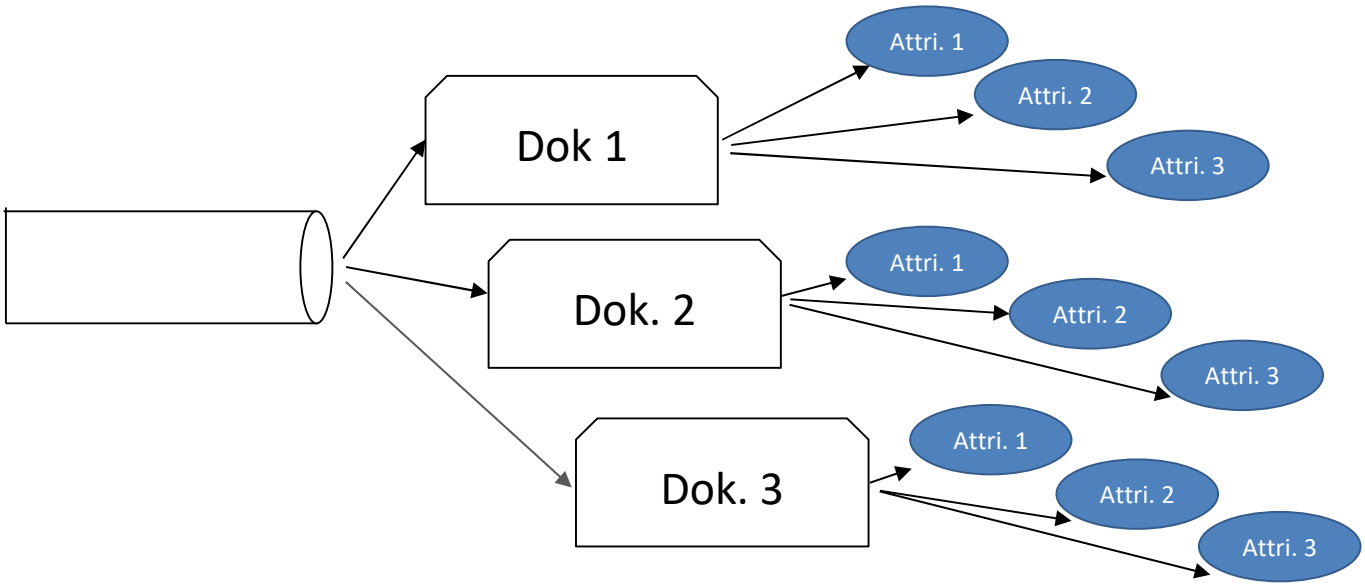
Pro
Nahtlose Abbildung objektorientierter Anwendungen, Methoden und Vererbung werden mitgespeichert, gut für komplexe Strukturen (CAD, Multimedia, technische Daten).

Kontra
Geringe Marktverbreitung, kaum Standardisierung und Werkzeuge, schwächere Ad-hoc-Auswertungen als SQL.

Praxis
Durchgesetzt haben sich objektrelationale Erweiterungen (PostgreSQL, Oracle) und ORM-Frameworks wie Hibernate oder Entity Framework; reine Systeme: db4o, ObjectDB, Versant.

DHBW Stuttgart

Dokumentenorientiertes Datenbankmodell (NoSQL)



Beispiel: ein Dokument (JSON)

```
{
  "kundennr": 4711,
  "name": "Meier",
  "adresse": { "ort": "Stuttgart",
               "plz": "70174" },
  "bestellungen": [
    { "artikel": "Tastatur", "menge": 2 },
    { "artikel": "Maus",     "menge": 1 }
  ]
}
```

Adresse und Bestellungen stehen im selben Dokument – kein Join über drei Tabellen nötig.

Prinzip

Daten werden als eigenständige Dokumente (JSON, BSON, XML) in Collections gespeichert. Jedes Dokument kann eigene Felder besitzen – das Schema liegt in der Anwendung, nicht in der Datenbank.
Zugriff über die Dokument-ID oder über Abfragen auf einzelne Felder; verschachtelte Strukturen und Listen sind direkt abbildbar.

Pro

Flexibles, leicht änderbares Schema, zusammengehörige Daten liegen beieinander, gute horizontale Skalierung (Sharding, Replikation).

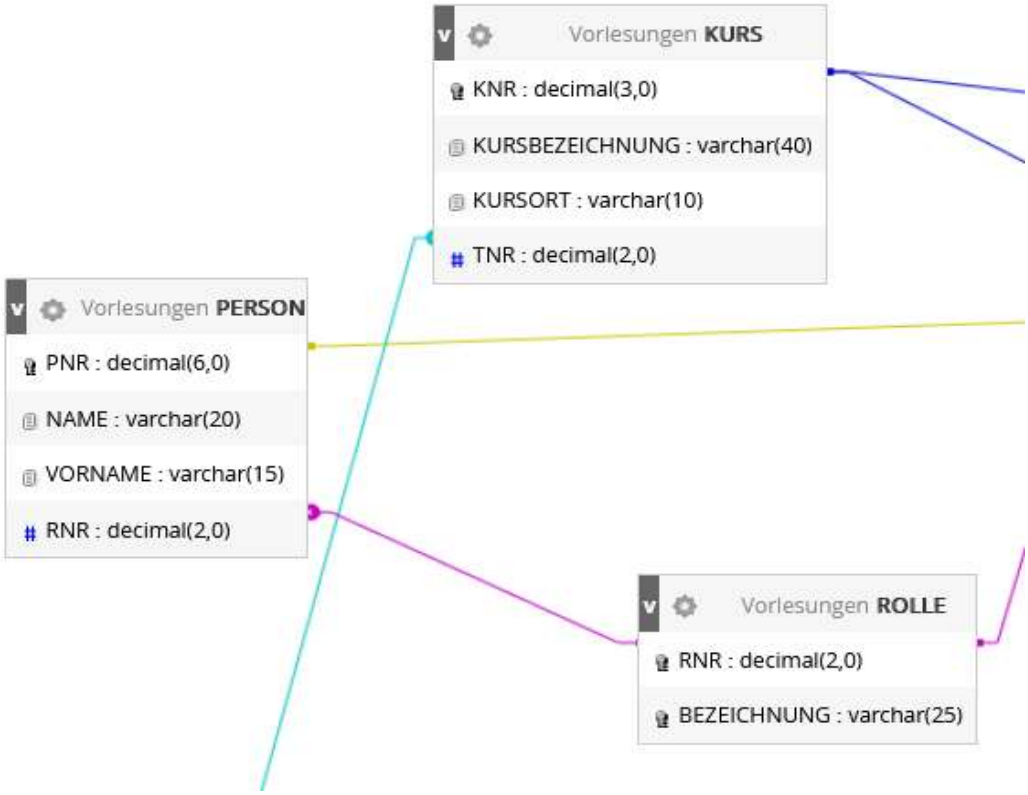
Kontra

Redundanz und Aktualisierungsaufwand bei mehrfach gespeicherten Daten, keine referentielle Integrität, Transaktionen über mehrere Dokumente nur eingeschränkt.

Typische Systeme

MongoDB, CouchDB, Amazon DocumentDB, Elasticsearch.

Relationales Datenbankmodell



Prinzip
Alle Daten liegen in Tabellen (Relationen): eine Zeile ist ein Tupel, eine Spalte ein Attribut mit festem Datentyp. Beziehungen werden nicht über Zeiger, sondern über Primär- und Fremdschlüssel hergestellt (im Beispiel PERSON.RNR → ROLLE.RNR).
Der Zugriff ist mengenorientiert und deklarativ über SQL; Grundlage ist die Relationenalgebra (Codd, 1970).

Pro
Einfaches, mathematisch fundiertes Modell, Standardsprache SQL, Integritätsbedingungen und Transaktionen (ACID), sehr große Werkzeug- und Wissensbasis.

Kontra
Festes Schema, Änderungen sind aufwändig; verschachtelte Strukturen müssen in mehrere Tabellen zerlegt werden (viele Joins); horizontale Skalierung schwieriger als bei NoSQL.

Typische Systeme
Oracle, Microsoft SQL Server, PostgreSQL, MySQL/MariaDB, IBM Db2, SQLite.

Hierarchisches Datenbankmodell

Fast nur noch historische Bedeutung, da in vielen Altinstallationen vorhanden.
Neben dem Netzwerkmodell erfolgreichstes Modell der 1. Generation.
Daten werden in einer festen Hierarchie abgespeichert.
Interessant für Banken, Versicherungen

Pro:
Übersichtlichkeit
Einfach aufgebaut
Schneller Lesezugriff

Kontra:
Starre Baumstruktur
Läßt keine Verknüpfungen zwischen den Bäumen zu
Nachträgliche Anpassungen mit hohem Aufwand verbunden

Netzwerkdatenbankmodell

Fast nur noch historische Bedeutung, da in vielen Altinstalleionen vorhanden.
Ist eine Erweiterung des hierarchischen Datenbankmodells, um Hierarchien zu beseitigen. Somit erreicht mal eine Darstellung der Verbindung der Daten. Datensätze sind verkettet.
Zugriff über Navigation(find, get, store – Befehle).

Pro:
Keine strenge Hierarchie
Mehrere Suchwege zum Datensatz möglich
Übersichtliche Abbildung netzartiger Stukturen

Kontra:
Wird unübersichtlich bei großen Datenmengen
Umsetzung komplexer Zusammenhänge aufwändig und kompliziert

DHBW Stuttgart

Objektorientiertes Modell

Daten werden im Sinne der Objektorientierung als Objekte verwaltet. Ziel ist es,
Objekte in ihrem Verhalten und ihren Eigenschaften in einer Datenbank zu verwalten
und zu speichern.
Ist eine Erweiterung des relationalen Datenbankmodells, um OO-Konzepte mit Hilfe von abstrakten Datentypen,
Spezialisierungshierarchien für Tabellen und Objektidentifkatoren.

Pro:
Unterstützung mehrdimensionaler Daten
Daten können flexibel repräsentiert werden
Mehrfachverwendung von Objekten

Kontra:
Performance
Wenig Schnittstellen vorhanden
Komplizierte Implementierung

Relationales Datenbankmodell

Tabellen sind Mengen von Tupeln (Datensätze) mit Beziehungen zueinander.
Tabellen sind flach und ungeschachtelt.
Findet in vielen Bereichen Anwendung: Warenwirtschaft, Finanzen, Controlling, Technik; Content-Management usw.

Pro:
Einfache Umsetzung
Flexibel erweiterbar
Daten bleiben unabhängig voneinander
Viele kommerzielle Anbieter

Kontra:
Fehler und –störanfällig
In manchen Bereichen weniger leistungsfähig
Mangelnde Datenintegrität

Dokumentenorientiertes Datenbankmodell (NoSQL-Datenbanken)

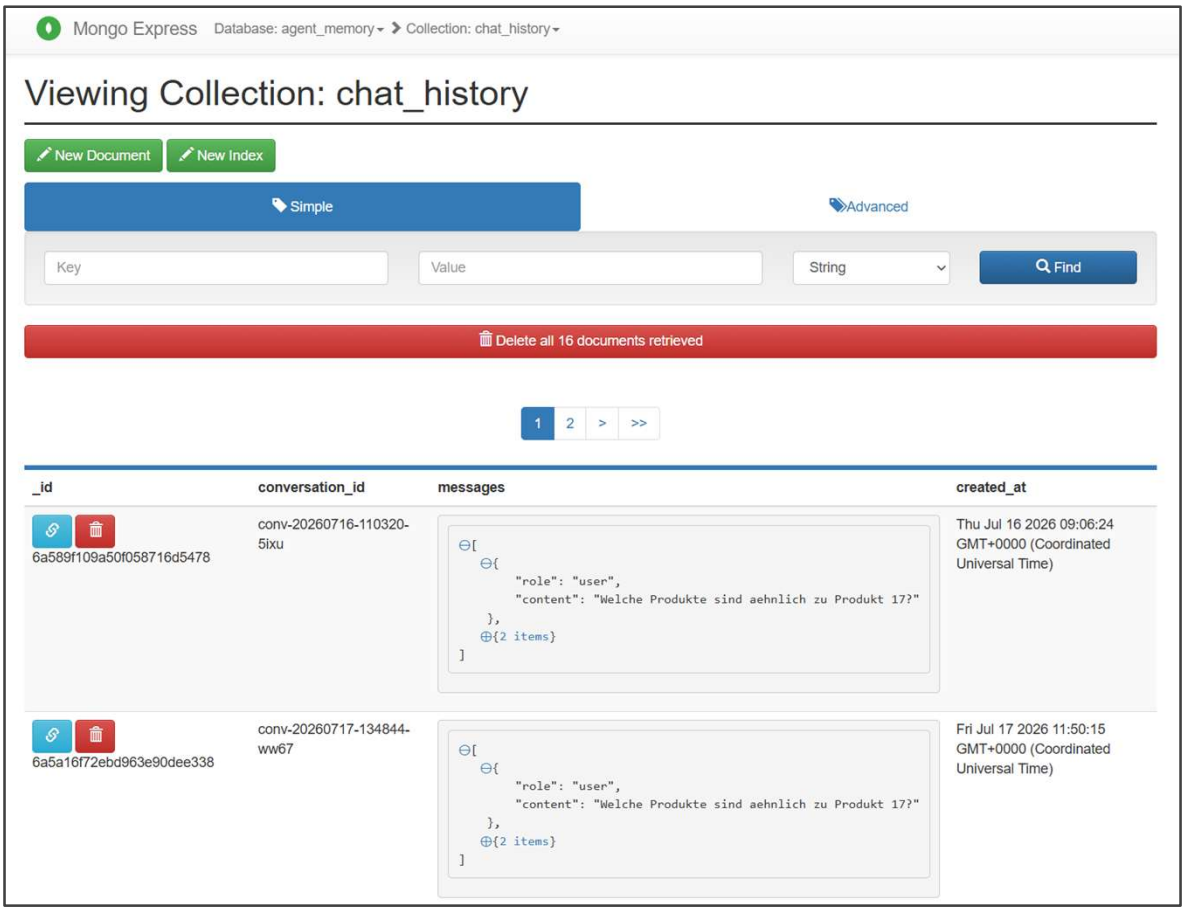
Eignet sich besonders für die Speicherung von Dokumenten, die stark vernetzt sind, zb. Web-Chat.
Diese NoSQL-Datenbank kann mehrer Key-Value-Paare speichern und verwalten. Bps: Lotus Notes. MongoDB, CouchDB (speichert im JSON-Format)

Pro:
Zentrale Speicherung zusammengehöriger Daten in einem Dokument (Bsp. Wikipedia)
Frei wählbare Struktur
Multimediale Ausrichtung
Ein Dokument kann mehrere Felder unterschiedlichen
Types haben (Text, Zahl, usw).
Integritätsbedingunen: referenzielle Int., Bereichsint.

Kontra:
Hoher Organisationsaufwand
Programmierkenntnisse erforderlich


```
⊖[
  ⊖{
      "role": "user",
      "content": "Welche Produkte sind aehnlich zu Produkt 17?"
   },
  ⊖{
      "role": "assistant",
      "content": "Keine belastbaren Hinweise"
   }
]
```


Praxisbeispiel: MongoDB (Mongo Express)



Graphdatenbankmodell (NoSQL)

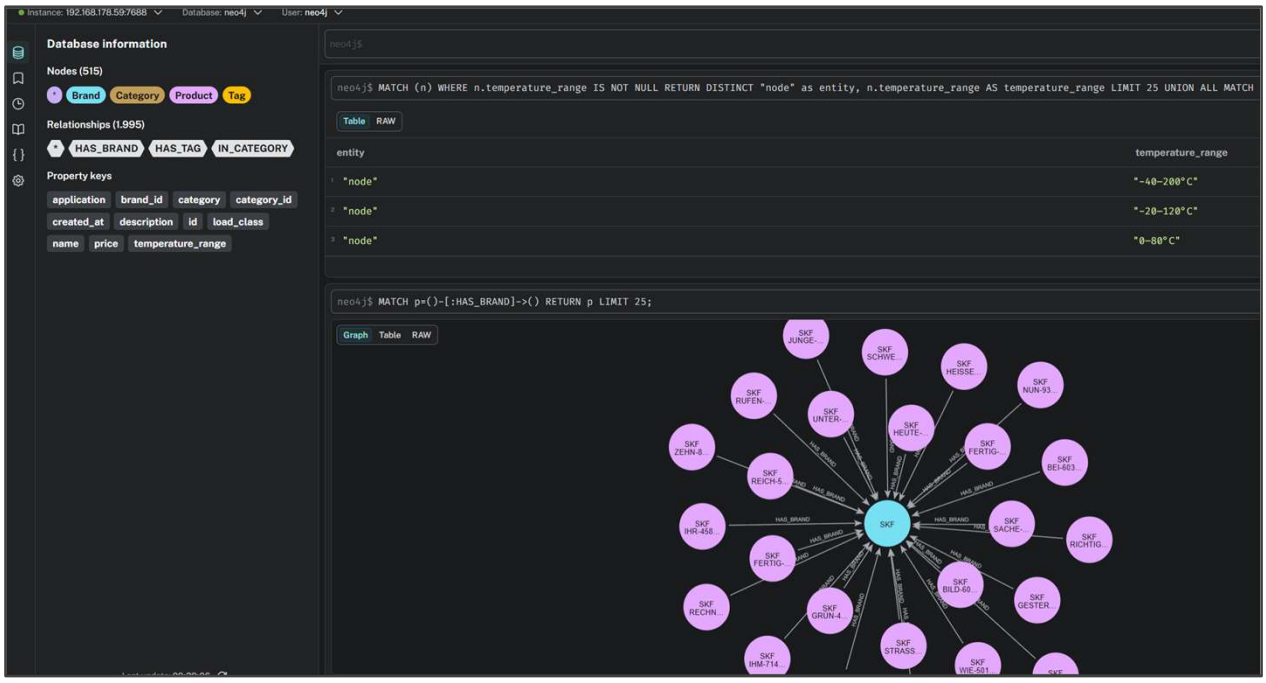
Daten werden als Graph gespeichert: Knoten (Entitäten), Kanten (Beziehungen) und Properties (Attribute an Knoten und Kanten).
Beziehungen sind gleichwertige Objekte und werden direkt gespeichert – kein Join über Fremdschlüssel nötig („index-free adjacency“).
Typische Systeme: Neo4j, ArangoDB, Amazon Neptune. Abfragesprachen: Cypher, Gremlin, SPARQL.

Beispiel (Cypher): MATCH (k:Kunde)-[:HAT_BESTELLT]->(a:Artikel) WHERE k.name = 'Meier' RETURN a.bezeichnung

Pro:
Beziehungsintensive Abfragen sehr performant (Empfehlungen, Netzwerke, Wege)
Flexibles Schema, Struktur kann mitwachsen
Mehrstufige Beziehungen ohne verschachtelte Joins abfragbar

Kontra:
Kein Standard wie SQL; Abfragesprachen herstellerabhängig
Schwächer bei aggregierenden Massenauswertungen über alle Datensätze
Horizontale Skalierung (Sharding) von Graphen ist aufwändig

Praxisbeispiel: Neo4j-Datenbank (Neo4j Browser)



Violette Knoten = Produkte, türkiser Knoten in der Mitte = Marke „SKF“. Die Pfeile sind Kanten vom Typ HAS_BRAND und zeigen jeweils vom Produkt auf die Marke – die Beziehung ist direkt gespeichert, kein Join über Fremdschlüssel nötig.

Weitere NoSQL-Modelle: Key-Value, Wide-Column, Vektor

Key-Value-Datenbanken
Einfachstes Modell: zu einem eindeutigen Schlüssel wird ein beliebiger Wert (Blob, JSON, Zähler) gespeichert. Zugriff nur über den Schlüssel.
Typische Systeme: Redis, Amazon DynamoDB, Riak. Einsatz: Sessions, Caches, Warenkörbe, Zähler.
Pro: extrem schnelle Lese- und Schreibzugriffe, einfach horizontal skalierbar. Kontra: keine Abfrage über Werte, keine Beziehungen.

Spaltenorientierte Modelle (Wide Column)
Daten werden nach Spaltenfamilien gruppiert; jede Zeile kann unterschiedliche Spalten besitzen. Optimiert für sehr große, verteilte Datenmengen.
Typische Systeme: Apache Cassandra, HBase, Google Bigtable. Einsatz: Zeitreihen, Logging, Sensordaten, Analytik.
Pro: lineare Skalierung über viele Knoten, schnelle Aggregation einzelner Spalten. Kontra: Abfragen müssen beim Datenmodell mitgedacht werden, Joins fehlen.

Vektordatenbanken
Speichern Embeddings – Zahlenvektoren, die Texte, Bilder oder Audio inhaltlich abbilden. Gesucht wird nach Ähnlichkeit (Kosinus-/Euklid-Distanz) statt nach exakter Übereinstimmung.
Typische Systeme: Pinecone, Milvus, Qdrant, pgvector (PostgreSQL-Erweiterung).
Einsatz: semantische Suche, Empfehlungen, Retrieval-Augmented Generation (RAG) für KI-Anwendungen.
Pro: Ähnlichkeitssuche in hochdimensionalen Daten, Näherungsverfahren (ANN) sehr schnell. Kontra: Ergebnisse sind approximativ, hoher Speicher- und Rechenbedarf.

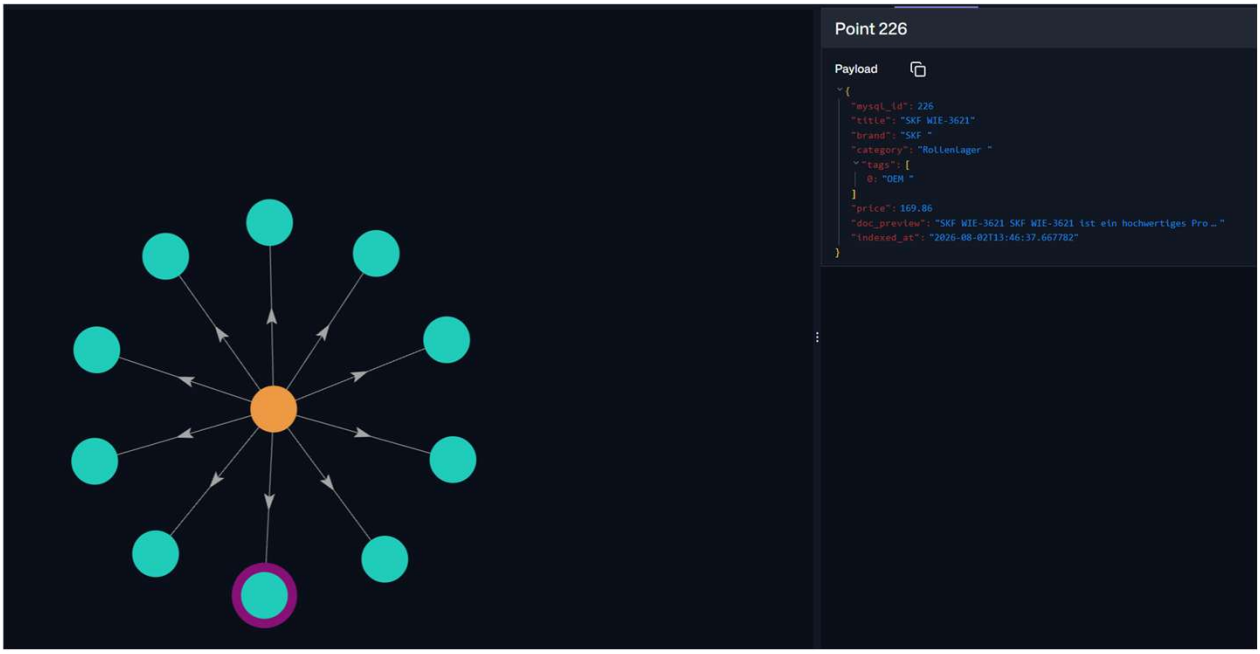
Vektordatenbankmodell (NoSQL)

Daten werden als Vektoren (Embeddings) gespeichert: Ein Modell überführt Text, Bild oder Audio in einen Punkt im hochdimensionalen Raum; an jedem Punkt hängt zusätzlich ein Payload mit Metadaten (z. B. Titel, Marke, Kategorie, Preis).
Abgefragt wird nicht exakt, sondern nach Ähnlichkeit: Gesucht werden die nächsten Nachbarn eines Anfragevektors über ein Distanzmaß (Kosinus, euklidisch, Skalarprodukt), beschleunigt durch einen ANN-Index (z. B. HNSW).
Typische Systeme: Qdrant, Milvus, Weaviate, Pinecone; auch als Erweiterung bestehender Systeme (pgvector für PostgreSQL).
Einsatz: semantische Suche, Empfehlungen, Duplikaterkennung, Bild-/Audioähnlichkeit, RAG für Sprachmodelle.

Pro:
Findet inhaltlich ähnliche Treffer statt nur exakter Wortübereinstimmungen
Sehr schnelle Nachbarschaftssuche auch bei Millionen von Vektoren
Filter auf Payload-Metadaten mit der Ähnlichkeitssuche kombinierbar

Kontra:
Ergebnisse sind approximativ – Trefferqualität (Recall) wird gegen Geschwindigkeit getauscht
Hoher Arbeitsspeicherbedarf, Qualität hängt vollständig vom Embedding-Modell ab
Keine klassischen Joins und Transaktionen; meist ergänzend zu einer Primärdatenbank

Praxisbeispiel: Qdrant-Datenbank (Qdrant Web UI, Graph-Ansicht)



Oranger Knoten = Ausgangspunkt der Suche, türkise Knoten = die k ähnlichsten Nachbarn im Vektorraum (hier k = 10). Wichtig: Die Kanten sind das Ergebnis einer kNN-Suche zur Laufzeit, keine in der Datenbank gespeicherten Beziehungen wie im Graphmodell. Rechts der Payload des markierten Punktes 226: SKF WIE-3621, Kategorie Rollenlager, Preis 169,86.

Datenbankmodelle im Vergleich

Modell	Struktur	Stärken	Schwächen	Typische Systeme
Hierarchisch	Baum, 1:n vom Vater zum Kind	Sehr schnelle Navigation, einfach	Keine n:m-Beziehungen, starr	IMS, Windows-Registry
Netzwerk	Netz aus verketteten Sätzen	n:m möglich, mehrere Zugriffswege	Unübersichtlich, navigierender Zugriff	IDMS, UDS
Relational	**Tabellen (Relationen), Schlüssel**	**Standard SQL, Konsistenz, ausgereift**	**Starres Schema, Joins bei vielen Ebenen**	**Oracle, PostgreSQL, MySQL**
Objektorientiert	Objekte mit Methoden und Vererbung	Nahe an der Programmierung	Kaum Verbreitung, wenig Standard	db4o, ObjectDB
Dokumentenorientiert	Dokumente (JSON/BSON) in Collections	Flexibles Schema, gut verschachtelbar	Redundanz, schwache Integritätsprüfung	MongoDB, CouchDB
Graph	Knoten, Kanten, Properties	Beziehungsabfragen sehr schnell	Kein SQL-Standard, Sharding schwierig	Neo4j, Neptune
Key-Value	Schlüssel → Wert	Sehr schnelle Zugriffe	Keine Suche über Werte	Redis, DynamoDB
Wide Column	Zeilen mit Spaltenfamilien	Lineare Skalierung, Analytik	Abfragen müssen vorab bedacht werden	Cassandra, HBase
Vektor	Embeddings als Zahlenvektoren	Ähnlichkeitssuche, KI/RAG	Approximative Treffer, ressourcenhungrig	Milvus, Qdrant, pgvector

Vertiefende Skripte zum Thema Vektordatenbanken

Skript (PDF)	Inhalt	Dateiname
Vektordatenbanken & Semantische Suche	Embeddings, Similarity Search, Qdrant, HNSW, RAG-Anbindung (mit Beispielen)	VektorDB.pdf
Vektorsuche vs. SQL-Suche	Indexaufbau und Online-Suche, Beispiel „Schiff“, Vergleichstabelle Qdrant vs. LIKE	Qdrant_vs_SQL_Suche.pdf
HNSW (Hierarchical Navigable Small World)	Von Brute Force O(n) zu HNSW O(log n): Ebenen, Greedy-Suche, Parameter m/ef/ef_construct	HNSW_Erklaerung.pdf

Download: tutor.kkessler.de/llm/download.php?file=downloads/ + Dateiname

Warum NoSQL? ACID, BASE und das CAP-Theorem

ACID – das Konsistenzversprechen relationaler Systeme
Atomicity: eine Transaktion wird vollständig oder gar nicht ausgeführt.
Consistency: jede Transaktion führt von einem konsistenten Zustand in einen konsistenten Zustand.
Isolation: parallele Transaktionen beeinflussen sich nicht.
Durability: bestätigte Änderungen bleiben auch nach einem Ausfall erhalten.

CAP-Theorem (Brewer)
In einem verteilten System sind Consistency (alle Knoten sehen dieselben Daten), Availability (jede Anfrage wird beantwortet) und Partition Tolerance (Betrieb trotz Netzausfall zwischen Knoten) nicht gleichzeitig vollständig erreichbar.
Da Netzpartitionen in verteilten Systemen unvermeidbar sind, fällt die Wahl praktisch zwischen CP (Konsistenz vor Verfügbarkeit, z. B. HBase) und AP (Verfügbarkeit vor Konsistenz, z. B. Cassandra, DynamoDB).
Damit erklärt das CAP-Theorem, warum verteilte Systeme ACID nicht uneingeschränkt halten können: AP-Systeme geben die starke Konsistenz auf und gehen zu BASE über.
Achtung: das C in CAP meint Linearisierbarkeit über Replikate (alle Knoten sehen den aktuellen Stand), das C in ACID die Einhaltung der Integritätsbedingungen innerhalb einer Transaktion.

BASE – der Gegenentwurf vieler NoSQL-Systeme
Basically Available: das System antwortet auch bei Teilausfällen.
Soft State: der Zustand darf zwischen Knoten vorübergehend abweichen.
Eventually Consistent: alle Replikate werden nach einiger Zeit konsistent.

Konsequenz für die Modellwahl
Transaktionskritische Anwendungen (Buchhaltung, Bestellungen) sprechen für relationale Systeme mit ACID; sehr große, verteilte, schreibintensive Datenmengen (Sensorik, Logs, Web-Skalierung) sprechen für NoSQL mit BASE.

Notiz: Aussprache und Auflösung der Akronyme
ACID – engl. „ässid“ (Säure): Atomicity, Consistency, Isolation, Durability
BASE – engl. „beiss“ (Base): Basically Available, Soft state, Eventually consistent
CAP – engl. „käp“, also „Käp-Theorem“: Consistency, Availability, Partition tolerance
NoSQL – „No-Ess-Ku-Ell“; gemeint ist „not only SQL“, nicht „kein SQL“
ACID und BASE sind bewusst als chemisches Gegensatzpaar gewählt: starke gegen abgeschwächte („eventual“) Konsistenz.

CAP am Beispiel: das letzte Stück im Lager

Onlineshop mit zwei Rechenzentren. Von einem Artikel ist genau 1 Stück auf Lager, beide Knoten halten eine Kopie des Bestands.



Option A: antworten

Hamburg verkauft das letzte Stück, ohne Stuttgart zu fragen.

Stuttgart verkauft im selben Moment möglicherweise dasselbe Stück.

Folge: zwei Kunden, ein Artikel. Die Kopien widersprechen sich.

→ Availability vor Consistency (AP)

Option B: verweigern

Hamburg antwortet „gerade nicht möglich“.

Es wird nichts verkauft, solange Stuttgart nicht erreichbar ist.

Folge: keine falschen Daten, aber der Shop ist nicht benutzbar.

→ Consistency vor Availability (CP)

Warum es keine dritte Option gibt

Partition Tolerance ist keine Wahl: sobald Daten auf mehreren Rechnern liegen, kann die Verbindung reißen. Im Partitionsfall bleibt nur C oder A – ein System mit allen drei Eigenschaften gibt es nicht.

„Partition“ heißt Netzspaltung, nicht Partitionierung: gemeint ist der Fehlerfall, dass Knoten sich gegenseitig nicht mehr erreichen – nicht das absichtliche Aufteilen von Daten auf mehrere Knoten (Sharding). Gleicher Wortstamm, anderer Begriff.

„CA“ heißt nicht verteilt: nur auf einem einzelnen Knoten kann keine Partition auftreten, dort sind C und A gleichzeitig möglich. Das ist die klassische relationale Datenbank mit ACID – dafür nicht über Knotengrenzen skalierbar.

Der Trade-off greift erst im Fehlerfall: solange das Netz intakt ist, liefert auch ein CP-System konsistente Antworten und Verfügbarkeit. CP bedeutet, im Partitionsfall die Verfügbarkeit zu opfern – nicht ständig.

Nach der Reparatur führt ein AP-System die Kopien zusammen: genau das beschreibt BASE mit „eventually consistent“.

NoSQL-Datenbanken „non-relational“

Daten sind nicht in Tabellen gespeichert, sondern in Spalten, Dokumenten oder Graphen.
Datenbanksprache ist nicht SQL.
Entstanden in der Web2.0-Welt (Facebook, Twitter etc. Mit Millionen Nutzern weltweit),
wo neue Anforderungen entstanden sind, die von relationalen Architekturen nicht hinreichend
erfüllt werden können (sehr schnell sehr große Daten speichern und wiederfinden, BigData fähig, hohe Performance,
einfache Replikation, hohe Skalierbarkeit usw.)

Wichtigeste Kategorien sind:

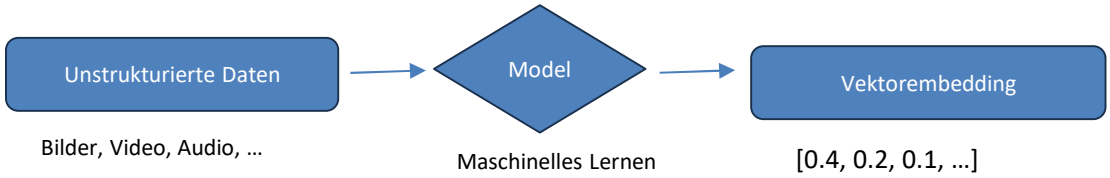
- **KeyValue-Datenbank:** verwaltet Tupel, die aus einem Schlüssel und einem Wert bestehen.
  Abfragen nur über den Key möglich. Wert ist ein Byte-Array. Vergleichbar mit einem Map in Java.

- **Spaltenorientierte Datenbank:** verwendet Tabellen, wo Anzahl der Spalten dynamisch ist. Sind quasi
  eine Verallgemeinerung des Key-Value-Konzepts. Verwendung für spaltenorientierte Funktionen (sum(), avg())

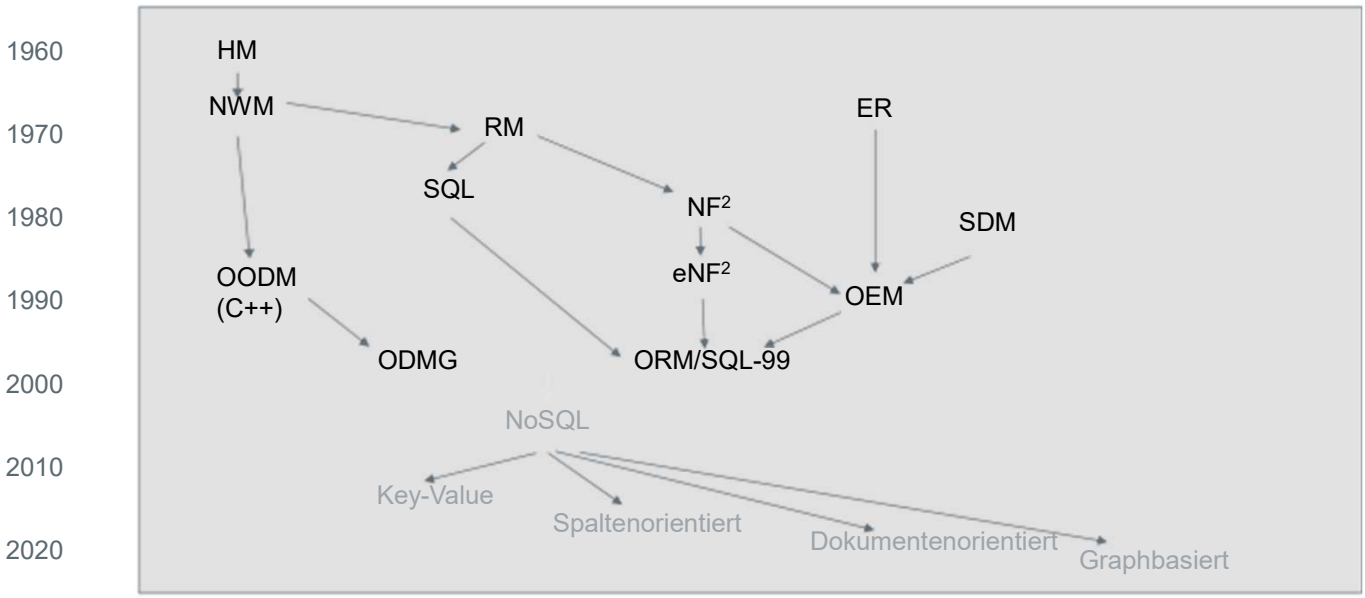
- **Graphen-Datenbanken:** spezialisiert auf die Verwaltung von Graphen und Knoten, Abfragen entlang der Graphen

- **Dokumentenorientierte Datenbanken:** s. oben, Suche auch im Value (Dokument), Key-Document-Datenbank

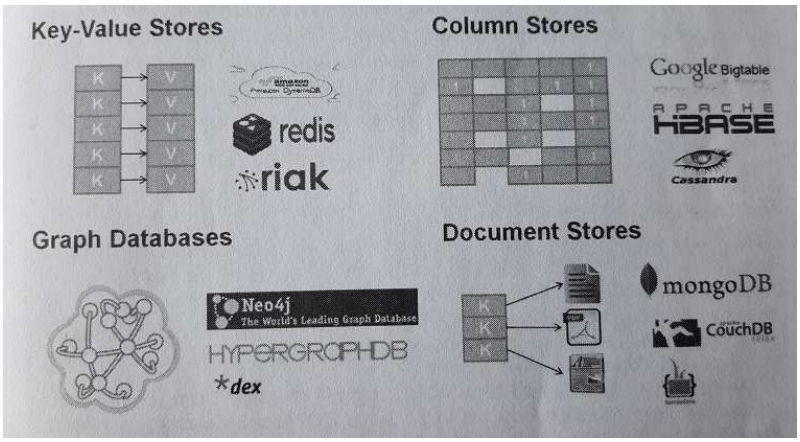
- **Vektordatenbanken:** Speichern Daten als hochdimensionale Vektoren ab und sind speziell für die effiziente Speicherung,
  Verwaltung und Abfrage von Vektordaten optimiert. Diese Art der Datenbank ist besonders nützlich für
  Anwendungen im Bereich maschinelles Lernen und künstliche Intelligenz, wo schnelle
  Ähnlichkeitssuchen und Berechnungen auf Vektoren erforderlich sind.

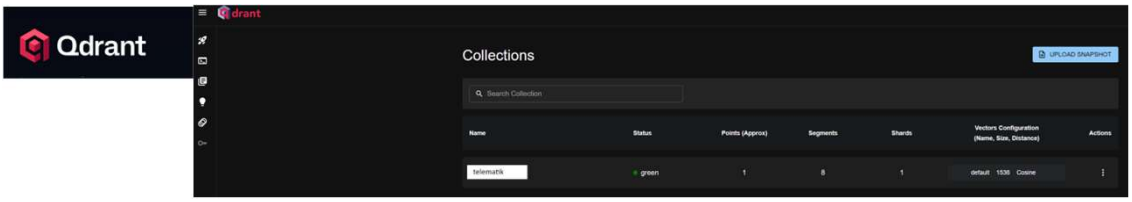


Datenmodelle – Historische Entwicklung

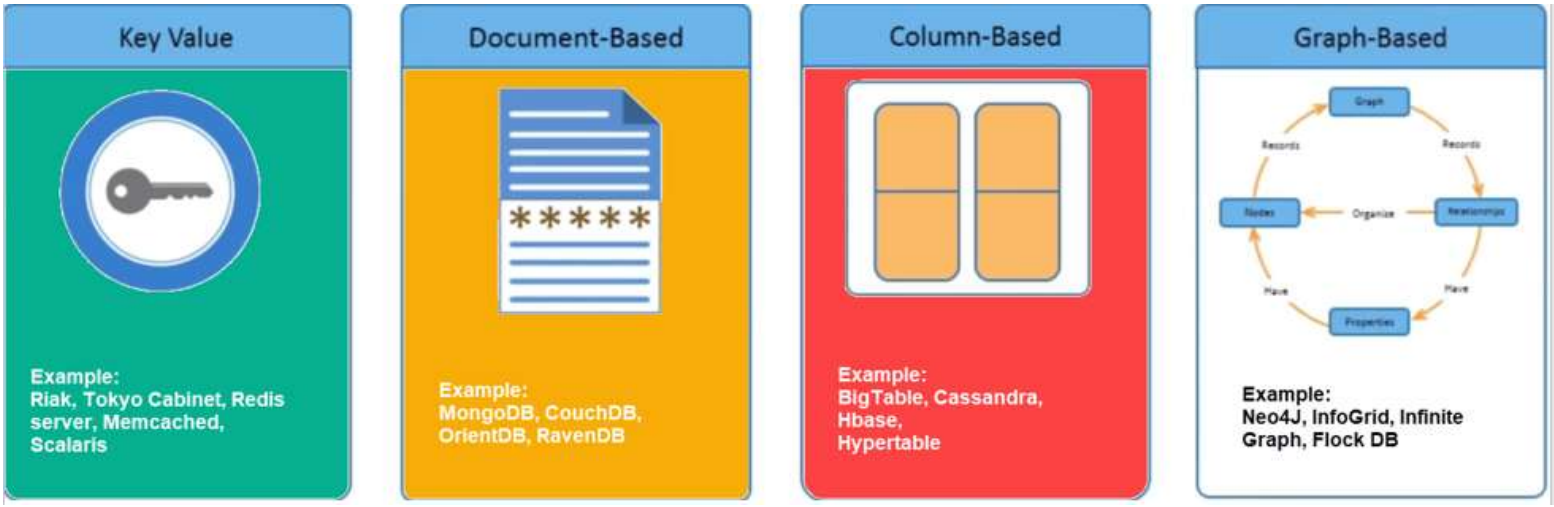


Beispiele von NoSQL-Datenbanken





Bildquelle: http:/www.nosql-database.org abgerufen am: 11.9.2022

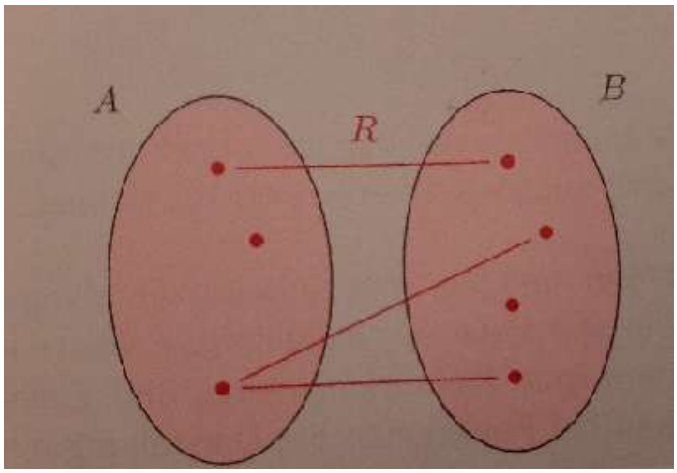


Bsp.: Google verwendet *BigTable,* eine spaltenorientierte DB.
Daten werden dabei in Zeilen abgelegt. Jede Zeile besteht aus einem Schlüssel und fast unbegrenzt vielen Spalten. Einige Spalten lassen sich zusammenfassen und auf einen Server ablegen (als verteilte Datenbank)

Relationen in der Mathematik

Definition: Eine Relation R zwischen den Mengan A und B legt fest, welche Elemente a aus A mit welchen Elementen b aus B in Beziehung stehen.

$$R \subseteq A_1 \times A_2 \times \ldots \times A_k$$



Beispiel: $A_1 = \{1, 2, 3\}$, $A_2 = \{a,b\}$

$R_1 = \{\}$ (leere Menge)

$R_2 = \{(1,a), (2,b)\}$

$R_3 = \{(1,a), (2,a), (3,a)\}$

$R_4 = D_1 \times D_2 = \{(1,a), (1,b), (2,a), (2,b), (3,a), (3,b)\}$

Beispiel aus der Vorlesungs-Datenbank (Schema „Vorlesungen“):

Es sei P die Menge der Personen (PERSON), K die Menge der Kurse (KURS), D die Menge der Dozenten (DOZENT).
Die Schlüsselspalten der Tabelle BELEGUNG bilden eine dreistellige Relation: BELEGUNG ⊆ P × K × D

BELEGUNG = {(4711, 12, 7), (4711, 15, 8), (4802, 12, 7), (4915, 21, 9)}

Jede Zeile der Tabelle liefert mit (PNR, KNR, DNR) genau ein Tupel dieser Relation;
DATUM ist ein zusätzliches beschreibendes Attribut und gehört nicht zum Schlüssel.

SQL: dreistellige Relation BELEGUNG

```
SELECT PNR, KNR, DNR FROM BELEGUNG;
SELECT PNR, KNR, DNR, DATUM FROM BELEGUNG WHERE DNR = 7;
SELECT COLUMN_NAME, COLUMN_KEY FROM information_schema.COLUMNS
 WHERE TABLE_SCHEMA = 'Vorlesungen' AND TABLE_NAME = 'BELEGUNG';
```

Jede Ergebniszeile ist ein Tupel der Relation; DATUM ist nur beschreibendes Attribut.

Vollständige, ausführbare Abfragen zu den Folien 34–39:
https://github.com/karkessler/dhbw-db-2627/blob/main/vorlesung/sql/Datenbanken2Uebungen.sql

Weiteres Beispiel:

Es sei G die Menge der Gebirge. G = {Alpen, Himalaya, Pyrenäen, Ural}

Es sei L die Menge der Länder. L = {Deutschland, Frankreich, Italien, Polen, Russland, Spanien}

Die Relation R_1 beschreibt den Zusammenhang zwischen den Gebirgen und den Ländern.
R_1 = {(Alpen, Deutschland), (Alpen, Frankreich), (Alpen, Italien), (Pyrenäen, Frankreich), (Pyrenäen, Spanien), (Ural, Russland)}

Beispiel aus der Vorlesungs-Datenbank:

Es sei D die Menge der Dozenten (Tabelle DOZENT, Schlüssel DNR). D = {7, 8, 9}
Es sei K die Menge der Kurse (Tabelle KURS, Schlüssel KNR). K = {12, 15, 21}

Die Relation $R_2 \subseteq D \times K$ beschreibt, welcher Dozent welchen Kurs hält:
R_2 = {(7, 12), (7, 21), (8, 15), (9, 21)}

Ein Dozent kann mehrere Kurse halten und ein Kurs von mehreren Dozenten gehalten werden:
eine n:m-Beziehung, die im Schema „Vorlesungen“ über die Tabelle BELEGUNG (PNR, KNR, DNR, DATUM) abgebildet wird.

SQL: n:m-Beziehung Dozent – Kurs

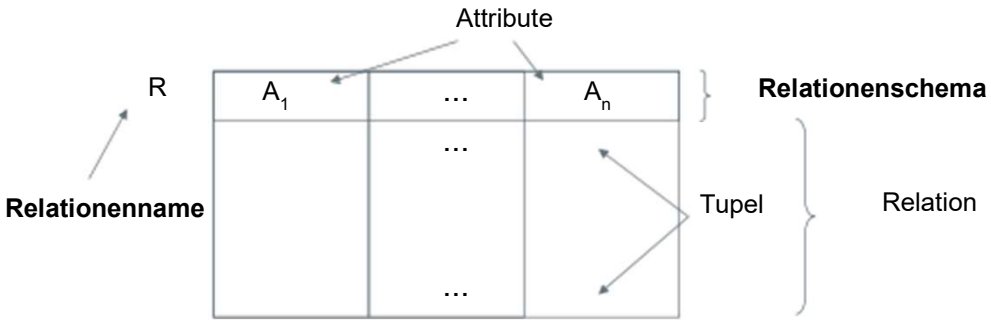
```
SELECT DISTINCT DNR, KNR FROM BELEGUNG ORDER BY DNR, KNR;
SELECT D.DNR, D.NAME, D.VORNAME, B.KNR
  FROM DOZENT D JOIN BELEGUNG B ON D.DNR = B.DNR;
SELECT DNR, COUNT(DISTINCT KNR) AS KURSE FROM BELEGUNG GROUP BY DNR;
```

BELEGUNG ist die Zwischentabelle, die die n:m-Beziehung auflöst.

Aufbau und Funktionen – Relationenmodell

Ein Modell für die konzeptuelle Ebene

Konzeptionell: „Ansammlung von Tabellen“ = Relationen



Beispiel: Relation DOZENT aus dem Schema „Vorlesungen“

DNR	STATUS	PNR	NAME	VORNAME	FIRMA
7	extern	4711	Braun	Martin	ACME AG
8	intern	4802	Keller	Anna	DHBW
9	extern	4915	Roth	Jens	Bosch GmbH

Relationenname: DOZENT
Relationenschema: DOZENT(DNR, STATUS, PNR, NAME, VORNAME, FIRMA)
Attribute: DNR, STATUS, PNR, … – Tupel: eine Zeile, z. B. (7, extern, 4711, Braun, Martin, ACME AG)
DNR ist Primärschlüssel (PRI), PNR ist Fremdschlüssel auf die Relation PERSON.

SQL: Relationenschema DOZENT

```
SELECT COLUMN_NAME, DATA_TYPE, COLUMN_KEY
  FROM information_schema.COLUMNS
 WHERE TABLE_SCHEMA = 'Vorlesungen' AND TABLE_NAME = 'DOZENT'
 ORDER BY ORDINAL_POSITION;
SELECT DNR, STATUS, PNR, NAME, VORNAME, FIRMA FROM DOZENT;
```

Attribute = Spalten, Tupel = Zeilen; DNR ist Primärschlüssel, PNR Fremdschlüssel auf PERSON.

Aufbau und Funktionen – Relationenmodell

Bibliotheksanwendung

BUCH



INV_NR	TITEL	ISBN	AUTOR
0001	Kommunikation-im-Netz	3-123	D.Reichardt
0002	Learn Java Now	3-239	S.Davis
0003	Javascript	3-111	D.Flanagan
...			
0196	Datenbanken	3-826	A.Heuer

AUSLEIHE

INV_NR	NAME
0001	G.Schröder
0196	D.Reichardt
...	
0202	J.W.Goethe

Integritätsbedingung:

- „Schlüsseleigenschaft“
  = Referenz muss **eindeutig** eine Zeile identifizieren
- Lokal + Global

Gleiches Prinzip in der Vorlesungs-Datenbank

DOZENT

DNR	NAME	FIRMA
7	Braun	ACME AG
8	Keller	DHBW
9	Roth	Bosch GmbH

BELEGUNG

PNR	KNR	DNR	DATUM
4711	12	7	2025-10-06
4711	15	8	2025-10-07
4802	12	7	2025-10-06

Integritätsbedingung: Jeder Fremdschlüsselwert muss eindeutig eine Zeile der referenzierten Relation identifizieren.
BELEGUNG.DNR → DOZENT.DNR | BELEGUNG.KNR → KURS.KNR | BELEGUNG.PNR → PERSON.PNR
KURS.TNR → THEMA.TNR | PERSON.RNR → ROLLE.RNR
KONTROLLE.KNR → KURS.KNR | KONTROLLE.RNR → ROLLE.RNR
Der Primärschlüssel von BELEGUNG ist zusammengesetzt: (PNR, KNR, DNR) – alle drei Spalten sind PRI.

SQL: referentielle Integrität prüfen

```
SELECT B.* FROM BELEGUNG B LEFT JOIN DOZENT D ON B.DNR = D.DNR WHERE D.DNR IS NULL;
SELECT K.* FROM KURS K LEFT JOIN THEMA T ON K.TNR = T.TNR WHERE T.TNR IS NULL;
SELECT P.* FROM PERSON P LEFT JOIN ROLLE R ON P.RNR = R.RNR WHERE R.RNR IS NULL;
```

Bleiben diese Ergebnisse leer, verweist jeder Fremdschlüsselwert auf eine existierende Zeile.

Aufbau und Funktionen – Anfrageoperationen

Selektion

SEL [NAME = ‚Horst'] (PERSON)
select * from PERSON where NAME = 'HORST'

PNR	NAME	VORNAME	RNR
232452	Horst	Petra	1

Projektion

PROJ [VORNAME, NAME] (PERSON)

select VORNAME, NAME from PERSON

VORNAME	NAME
Peter	Schneider
Kurt	Oswald
Petra	Horst
Franz	Maier
Hans	Meier

Verbund

PROJ [KURSBEZEICHNUNG, KURSORT] (KURS)
JOIN
SEL [THEMENGEBIET= ‚Analysis'] (THEMA)

select k.KURSBEZEICHNUNG, k.KURSORT from KURS k inner join THEMA t ON t.TNR = k.TNR where t.THEMENGEBIET = 'Analysis'

KURSBEZEICHNUNG	KURSORT
Analysis 3	1010.4.08
Algebraische Gleichungen	1010.2.05

SQL: Anfrageoperationen im Schema „Vorlesungen"

Selektion – Zeilen auswählen:

```
SELECT * FROM KURS WHERE KURSORT = 'Stuttgart';
```

Projektion – Spalten auswählen:

```
SELECT KNR, KURSBEZEICHNUNG FROM KURS;
```

Join – Tabellen verknüpfen:

```
SELECT P.NAME, P.VORNAME, K.KURSBEZEICHNUNG, B.DATUM
  FROM BELEGUNG B
  JOIN PERSON P ON B.PNR = P.PNR
  JOIN KURS   K ON B.KNR = K.KNR
 ORDER BY P.NAME, K.KURSBEZEICHNUNG;
```

Jede Operation liefert wieder eine Relation – daher sind Anfragen beliebig kombinierbar.

Aufbau und Funktionen – Sprachen und Sichten

SQL

```
SELECT  BUCH.INV_NR, TITEL, NAME
FROM      BUCH, AUSLEIHE
WHERE   NAME = ‚D.Reichardt‘ AND
            BUCH.INV_NR = AUSLEIHE.INV_NR
```

Benutzungssichten

Dirk Reichardts ausgeliehene Bücher :

Neue Tabelle : BUCHLISTE

BUCHLISTE := PROJ [INV_NR, TITEL] (BUCH)
JOIN
SEL [NAME = „D.Reichardt“] (AUSLEIHE)

SQL: lesende Anfragen auf dem Schema „Vorlesungen“

Schema über das Data Dictionary abfragen:

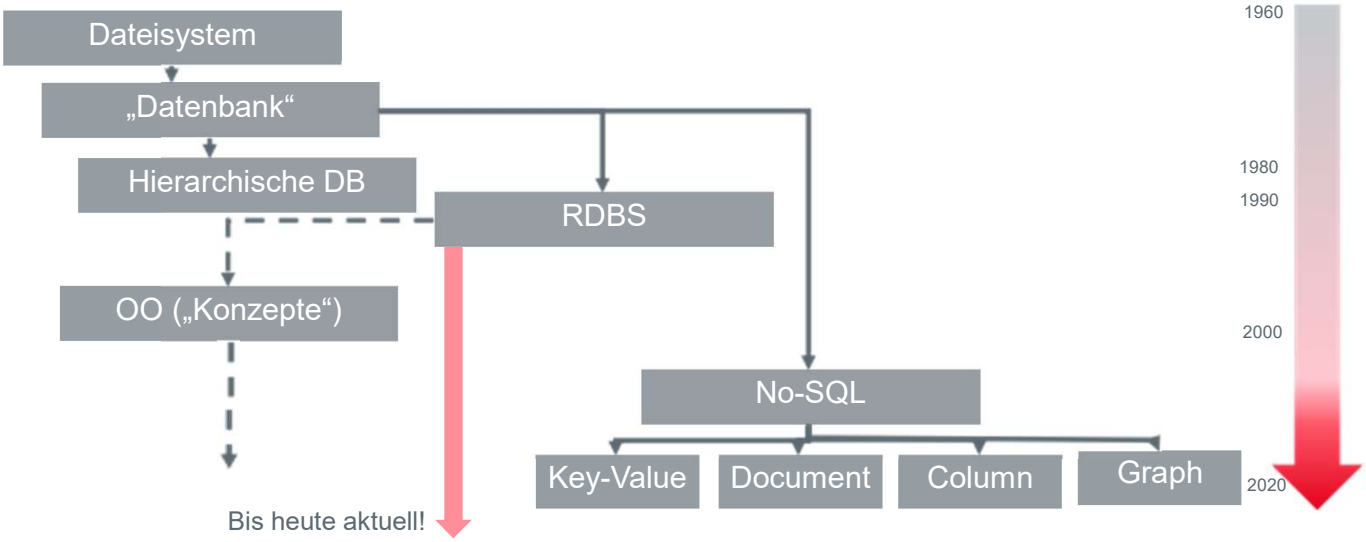
```
SELECT TABLE_NAME, COLUMN_NAME, DATA_TYPE, COLUMN_KEY
  FROM information_schema.COLUMNS
 WHERE TABLE_SCHEMA = 'Vorlesungen'
 ORDER BY TABLE_NAME, ORDINAL_POSITION;
```

Auswertung über mehrere Relationen:

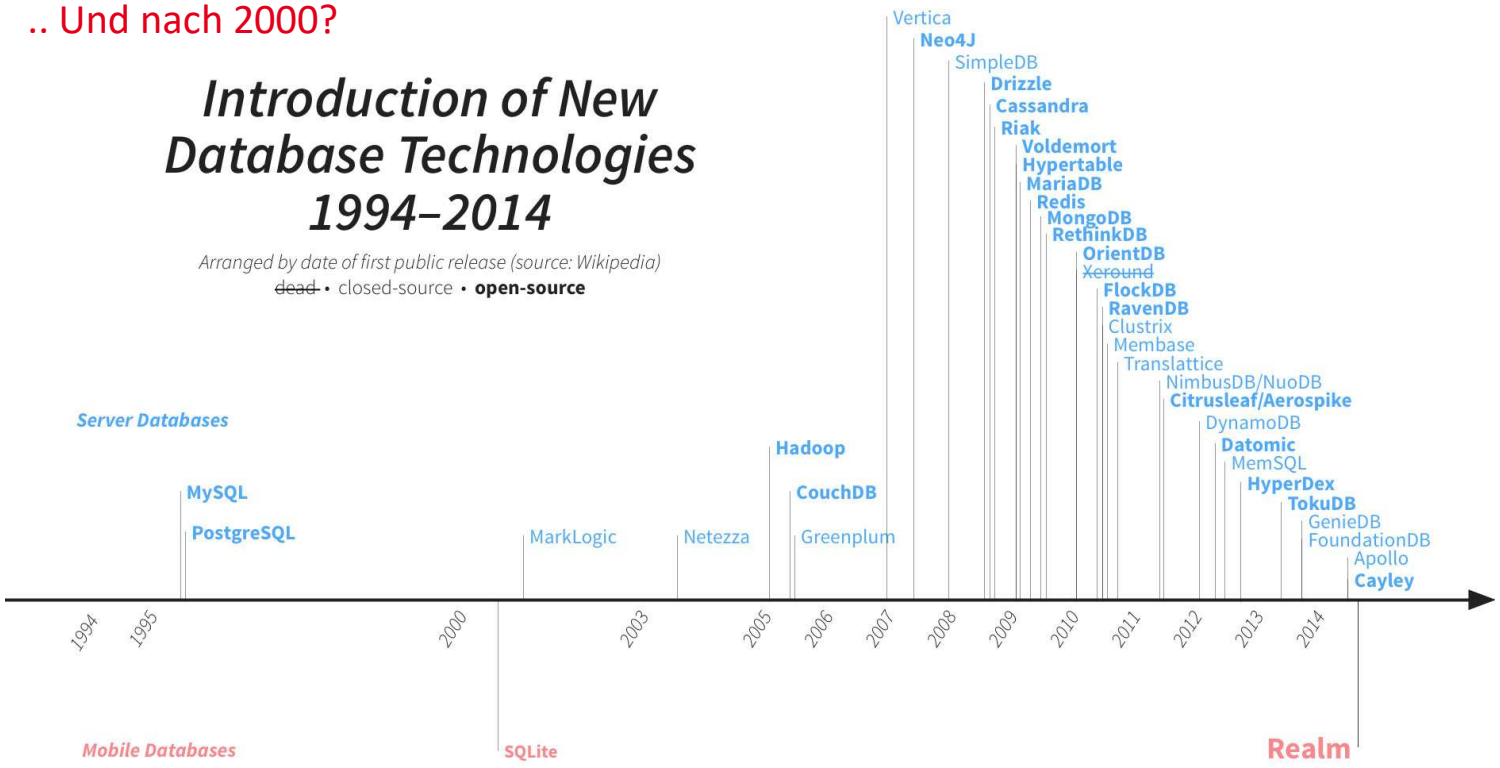
```
SELECT P.NAME, K.KURSBEZEICHNUNG, T.THEMENGEBIET, B.DATUM
  FROM BELEGUNG B
  JOIN PERSON P ON B.PNR = P.PNR
  JOIN KURS   K ON B.KNR = K.KNR
  JOIN THEMA  T ON K.TNR = T.TNR
 WHERE T.THEMENGEBIET = 'Datenbanken';
```

Alle Anfragen sind rein lesend – das bestehende Schema und die Daten bleiben unverändert.

Datenmodelle – was kommt jetzt?

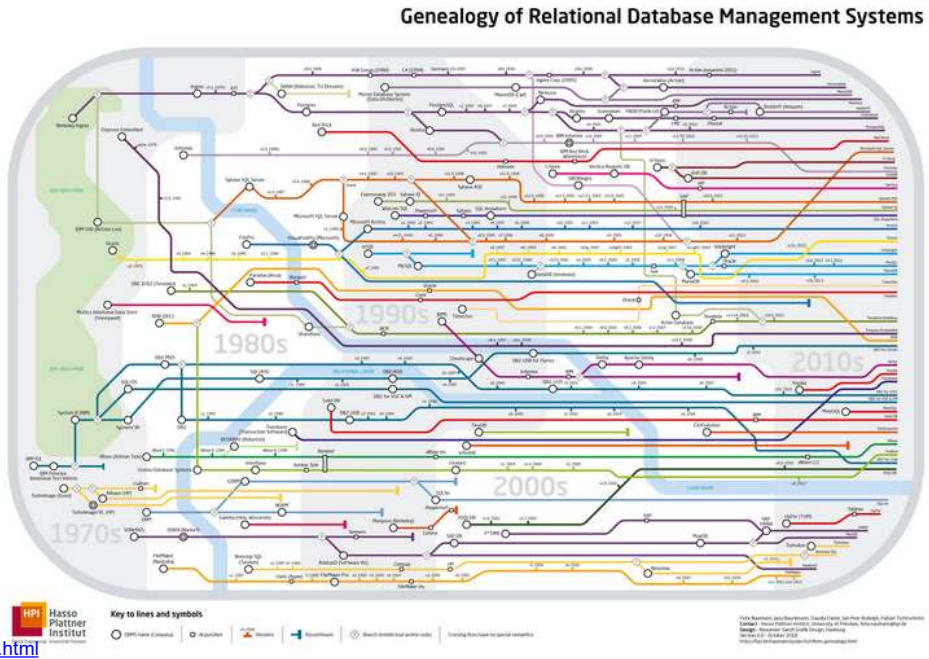


.. Und nach 2000?



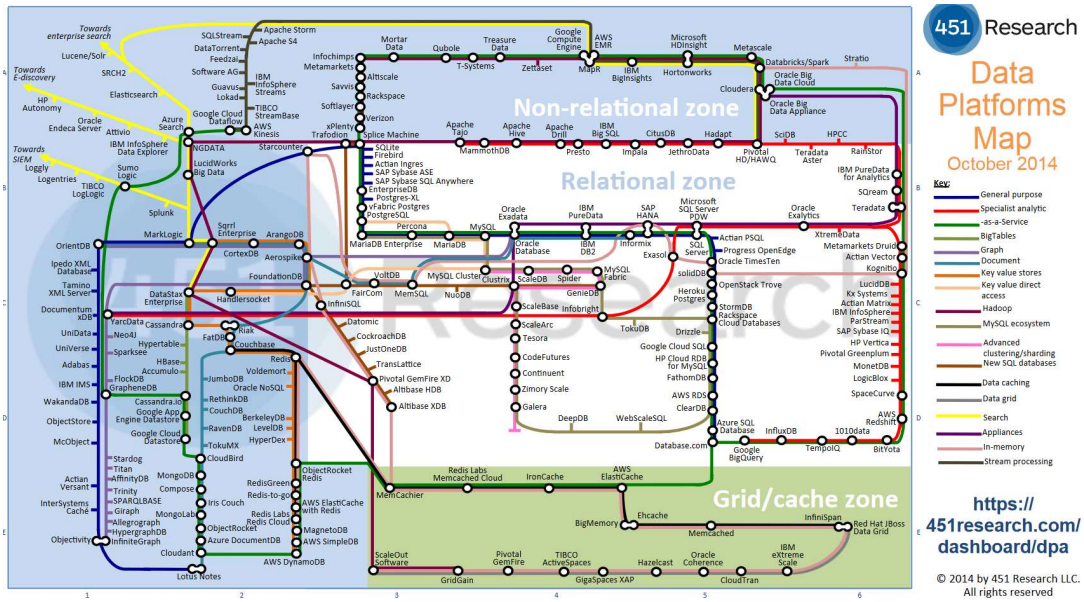
Bildquelle: Database Timeline: introduction of New Database Technologies 1994-2014, https://www.pinterest.de/pin/793829871788458949/, 21.1.2020

Übersicht über „relationale“ Datenbanksysteme



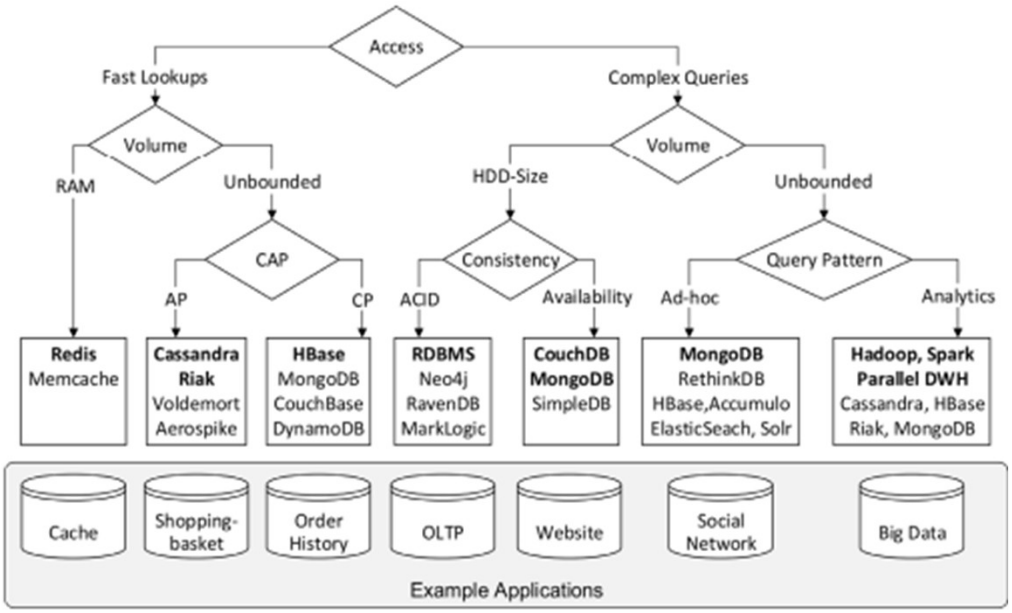
https://hpi.de/naumann/projects/rdbms-genealogy.html

Übersicht über „alle“ Datenbank systeme



https://451research.com/images/Marketing/dataplatformsmapoctober2014.pdf

NoSQL



Bildquelle: https://images.app.goo.gl/2497nX6TMEaCjXk4A, abgerufen am: 4.3.20

Vielen Dank für Ihre Aufmerksamkeit!